

Lenguajes de Programacion

Alto Nivel y Bajo Nivel

Estudiante:

Angel Fernando Calero Maldonado

Universidad Estatal de Milagro

UNEMI
UNIVERSIDAD ESTATAL DE MILAGRO

Que es un Lenguaje de Programacion?

Un **lenguaje de programacion** es un sistema formal de instrucciones que permite expresar algoritmos para que una computadora realice tareas especificas. Segun su cercania al hardware, se clasifican principalmente en:

Alto Nivel

Usa una sintaxis cercana al lenguaje humano. Facilita la lectura, escritura y mantenimiento del codigo, ocultando gran parte de los detalles internos del hardware.

Bajo Nivel

Se comunica de manera directa o casi directa con el procesador. Permite mayor control sobre memoria, registros e instrucciones, pero exige conocimientos tecnicos mas profundos.

“A mayor abstraccion, mayor facilidad para el programador; a menor abstraccion, mayor control sobre la maquina.”

Un **lenguaje de alto nivel** esta disenado para que el programador pueda resolver problemas usando instrucciones comprensibles y estructuras logicas cercanas al razonamiento humano.

Caracteristicas principales

- ▶ **Abstraccion del hardware:** oculta registros, direcciones de memoria y detalles internos del procesador.
- ▶ **Portabilidad:** el mismo codigo puede adaptarse a diferentes sistemas operativos o arquitecturas.
- ▶ **Productividad:** permite desarrollar programas con menos lineas de codigo y en menor tiempo.
- ▶ **Bibliotecas y frameworks:** ofrece herramientas listas para crear aplicaciones, paginas web, sistemas de datos o inteligencia artificial.

Ejemplos: Python, Java, JavaScript, C#, Ruby, PHP.

Un **lenguaje de bajo nivel** trabaja muy cerca del lenguaje maquina. Sus instrucciones se relacionan directamente con operaciones que el procesador puede ejecutar, por lo que ofrece mayor control y menor abstraccion.

Lenguaje Maquina

Es el nivel mas bajo. Esta formado por instrucciones binarias, es decir, secuencias de 0 y 1 que la CPU interpreta directamente.

Ejemplos: NASM, MASM, GAS, ARM Assembly, x86 Assembly.

Lenguaje Ensamblador

Usa instrucciones mnemotecnicas como MOV, ADD o INT. Debe traducirse a lenguaje maquina mediante un programa llamado ensamblador.

✓ Ventajas

- ▶ Facilita la lectura y escritura del código.
- ▶ Reduce el tiempo de desarrollo.
- ▶ Permite crear aplicaciones complejas con mayor rapidez.
- ▶ Es más fácil de mantener, corregir y actualizar.
- ▶ Tiene amplia documentación, comunidades y bibliotecas.

✗ Desventajas

- ▶ Puede ser menos eficiente que el código de bajo nivel.
- ▶ Consume más memoria por sus capas de abstracción.
- ▶ Ofrece menor control directo del hardware.
- ▶ Depende de intérpretes, compiladores o máquinas virtuales.
- ▶ No siempre es adecuado para sistemas críticos o embebidos.

✓ Ventajas

- ▶ Permite controlar directamente el hardware.
- ▶ Genera programas altamente eficientes.
- ▶ Optimiza el uso de memoria y procesador.
- ▶ Es fundamental en sistemas operativos y controladores.
- ▶ Resulta útil en sistemas embebidos y dispositivos IoT.

✗ Desventajas

- ▶ Tiene una curva de aprendizaje elevada.
- ▶ Su código es más difícil de leer y depurar.
- ▶ Depende de la arquitectura del procesador.
- ▶ Aumenta el riesgo de errores críticos.
- ▶ El desarrollo suele ser más lento y costoso.

El siguiente programa imprime un mensaje en pantalla utilizando **Python**, un lenguaje de alto nivel caracterizado por su sintaxis clara:

```
# Programa en Python - Alto Nivel
# Objetivo: imprimir un mensaje en pantalla

mensaje = "Hola, Mundo!"
print(mensaje)
```

Observacion

Con una sola instruccion principal, `print()`, el programador puede mostrar texto en pantalla. Python gestiona internamente detalles como la salida estandar, la codificacion del texto y la comunicacion con el sistema operativo.

El mismo procedimiento puede representarse en **pseudocodigo**, una forma estructurada de describir algoritmos sin depender de un lenguaje real:

```
INICIO
  DEFINIR mensaje COMO CADENA
  mensaje <- "Hola, Mundo!"
  ESCRIBIR mensaje
FIN
```

Proposito del Pseudocodigo

El pseudocodigo permite planificar la logica de un algoritmo antes de llevarlo a un lenguaje de programacion. Por eso se utiliza frecuentemente en la ensenanza de programacion y resolucion de problemas.

La misma tarea en **ensamblador** requiere declarar datos, cargar registros del procesador e invocar una llamada al sistema:

```
section .data
    msg db "Hola, Mundo!", 0x0A
    len equ $ - msg

section .text
    global _start

_start:
    mov eax, 4      ; sys_write
    mov ebx, 1      ; salida estandar
    mov ecx, msg    ; direccion del mensaje
    mov edx, len    ; longitud del mensaje
    int 0x80        ; llamada al kernel

    mov eax, 1      ; sys_exit
    xor ebx, ebx    ; codigo de salida 0
    int 0x80
```

Por debajo del ensamblador se encuentra el **lenguaje maquina**. Este se representa mediante codigos binarios o hexadecimales que el procesador puede ejecutar directamente:

```
Instruccion: int 0x80
Hexadecimal : CD 80
Binario : 11001101 10000000

Instruccion: mov eax, 4
Hexadecimal : B8 04 00 00 00
Binario : 10111000 00000100 00000000 00000000 00000000

Instruccion: mov ebx, 1
Hexadecimal : BB 01 00 00 00
Binario : 10111011 00000001 00000000 00000000 00000000
```

Reflexion

La CPU no interpreta directamente Python ni pseudocodigo. Todo programa debe terminar traducido a instrucciones de maquina para poder ejecutarse.

Tarea: Imprimir “Hola, Mundo!” en pantalla

Alto Nivel

```
print("Hola, Mundo!")
```

Simple y legible

Ensamblador

Declarar datos

Cargar registros

Indicar longitud

Llamar al kernel

Mayor control tecnico

Maquina

B8 04 00 00 00

BB 01 00 00 00

CD 80 ...

Bits y bytes

La diferencia principal esta en el nivel de abstraccion: mientras Python simplifica la tarea, el ensamblador exige controlar los detalles internos.

Lenguajes de Alto Nivel – Python, Java, JavaScript, C#

Lenguajes de Nivel Medio – C, C++

Lenguaje Ensamblador – NASM, MASM, GAS

Lenguaje Maquina – Codigo Binario (0s y 1s)

Hardware – Procesador (CPU)

Mayor
abstraccion



Se usa Alto Nivel cuando...

- ▶ Se crean aplicaciones web, móviles o de escritorio.
- ▶ Se necesita desarrollar rápido.
- ▶ Se trabaja con datos, inteligencia artificial o automatización.
- ▶ La facilidad de mantenimiento es importante.
- ▶ El proyecto debe funcionar en diferentes plataformas.

Se usa Bajo Nivel cuando...

- ▶ Se programan sistemas operativos o kernels.
- ▶ Se desarrollan controladores de hardware.
- ▶ Se trabaja con microcontroladores o sistemas embebidos.
- ▶ Se requiere máxima eficiencia.
- ▶ Se analiza el comportamiento interno de un programa.

Sintesis

- ▶ Los lenguajes de **alto nivel** priorizan la facilidad de uso, la productividad y la claridad del código.
- ▶ Los lenguajes de **bajo nivel** priorizan el rendimiento, el control del hardware y la eficiencia.
- ▶ Ambos tipos son necesarios: cada uno responde a necesidades distintas dentro del desarrollo de software.
- ▶ Comprender el bajo nivel ayuda a escribir programas de alto nivel mas eficientes y conscientes del uso de recursos.

“Un buen programador no solo escribe instrucciones; tambien comprende como esas instrucciones llegan a ejecutarse en la maquina.”

- ▶ Aho, A. V., Lam, M. S., Sethi, R., & Ullman, J. D. (2006). *Compilers: Principles, Techniques, and Tools*. Pearson.
- ▶ Bryant, R. E., & O'Hallaron, D. R. (2016). *Computer Systems: A Programmer's Perspective*. Pearson.
- ▶ Patterson, D. A., & Hennessy, J. L. (2021). *Computer Organization and Design: The Hardware/Software Interface*. Morgan Kaufmann.
- ▶ Sebesta, R. W. (2016). *Concepts of Programming Languages*. Pearson.
- ▶ Stallings, W. (2019). *Computer Organization and Architecture*. Pearson.
- ▶ Tanenbaum, A. S., & Bos, H. (2015). *Modern Operating Systems*. Pearson.
- ▶ Van Rossum, G., & Drake, F. L. (2009). *The Python Language Reference Manual*. Network Theory Ltd.

www.unemi.edu.ec

Gracias

UNEMI
UNIVERSIDAD ESTATAL DE MILAGRO